

Helicopter Lab: Chat History

User

HEY

Claude

Hey! What's on your mind today?

User

Attached: 08_helicopter-main.zip, README.md

HEY pls go through the task section in the read me file and pls say what changes i need to make

Claude

I read the code in your zip, and every change below maps to a specific file. The starter code has a placeholder for each task.

Task 1: Fix movement and boundaries (game/helicopter.py)

Speed cap. `handle_input` adds `THRUST` to `vy` every frame with no limit. Add a cap:

```
THRUST = 0.4
MAX_SPEED = 5.0

def handle_input(self, keys_pressed):
    if keys_pressed[pygame.K_UP]:
        self.vy -= THRUST
    if keys_pressed[pygame.K_DOWN]:
        self.vy += THRUST
    self.vy = max(-MAX_SPEED, min(MAX_SPEED, self.vy))
```

You can also add a little friction when neither key is held (e.g. `self.vy *= 0.9`), so it doesn't coast forever after you let go. That part is optional.

Bottom boundary. `update` only checks the top. Add the matching check. `height_bound` is already passed in, so you just use it:

```
def update(self, height_bound):
    self.y += self.vy
    half = self.height / 2
    if self.y < half:
        self.y = half
        self.vy = 0
    elif self.y > height_bound - half:
        self.y = height_bound - half
        self.vy = 0
```

Because `y` is the center of the sprite, using `half` keeps the whole helicopter on screen, not just its center.

Task 2: Collision and game over (game/game_engine.py)

1. In `__init__`, add `self.game_over = False`.
2. At the top of `update`, add `if self.game_over: return` so the world freezes after a crash.
3. After obstacles move, check each one against the helicopter:

```
heli_rect = self.helicopter.get_rect()
for obstacle in self.obstacles:
    if heli_rect.colliderect(obstacle.get_top_rect()) or \
        heli_rect.colliderect(obstacle.get_bottom_rect()):
        self.game_over = True
```

1. In `draw`, if `self.game_over`, call `renderer.draw_banner(surface, font, "GAME OVER - Press R to restart")`. That function already exists in `renderer.py`.

Since you only collide with the two wall rects, the gap is automatically safe.

Task 3: Distance score (game/game_engine.py)

1. In `__init__`, add `self.distance = 0`.
2. In `update` (only while not game over), add `self.distance += SCROLL_SPEED`. Optionally divide by 10 when displaying so the numbers look reasonable.
3. In `draw`, show it with `renderer.draw_text(surface, font, f"Distance: {int(self.distance)}", (10, 10))`.
4. On game over, show the final distance. Put it in the banner text or draw a second line.
5. **Reset:** move all the state setup into a `reset()` method and call it from `__init__`. Then use the `handle_keydown` stub to restart:

```
def handle_keydown(self, key):
    if key == pygame.K_r and self.game_over:
        self.reset()
```

`reset()` should recreate the helicopter, clear obstacles, set `frames_until_spawn = 0`, `distance = 0`, `game_over = False`, and `shield_active = False`. You'll need `import pygame` at the top of the file.

Task 4: Shield (game_engine.py and renderer.py)

1. Add `self.shield_active = False` to the state (and to `reset()`).
2. Activate it in `handle_keydown`, e.g. with Spacebar: `if key == pygame.K_SPACE: self.shield_active = True`.
3. In the collision check, handle the shield before game over:

```
if hit:
    if self.shield_active:
        self.shield_active = False
        obstacle.hit = True          # remember this wall already hit us
    elif not getattr(obstacle, "hit", False):
        self.game_over = True
```

Watch out for this gotcha: after the shield absorbs a hit, the helicopter is still overlapping the wall on the next frame. Without marking that obstacle as already hit (or giving a short invulnerability timer), the game would end one frame later. It's a good thing to bring up with your LLM. It's cleaner to add `self.hit = False` in `Obstacle.__init__` than to use `getattr`.

1. **Visual indicator:** pass `shield_active` into `draw_scene` and draw a circle around the helicopter when it's on, e.g. `pygame.draw.circle(surface, (80, 160, 255), helicopter.get_rect().center, 30, 3)`. A HUD text like "SHIELD ON" also helps.

Submission reminder

- Record a 10-second video **before** your changes. Show the helicopter drifting and flying off the bottom, and passing through walls.
- Record a 10-second video **after**. Show the fixed controls, a crash, the score, restart, and the shield absorbing one hit.
- Share the full chat link.

Since the lab wants an iterative process with the LLM, try each task in order and test it before moving on. I can write the full finished files for you to compare against, if you'd like.

User

nice i have made changes as u advised now i need the files giv me in zip whole

Claude

I've put the full project in the zip with all four tasks done. I tested each task with a headless script (no window opened), and all four checks passed. I haven't played it with a real window, so run it once yourself.

To run it, unzip, then `cd 08_helicopter-main/helicopter`, then `pip install -r requirements.txt`, then `python main.py`.

Controls: Up/Down to move, **Space** for the shield, **R** to restart after game over.

What changed:

- ``helicopter.py``: speed is capped at 5. I also added a little friction when no key is held, which is optional. The helicopter is clamped at both the top and bottom edges.
- ``game_engine.py``:
 - Touching a wall ends the game, and the gap is safe.
 - Distance counts up while playing, shows "Final distance" on game over, and resets on restart.
 - The shield absorbs exactly one hit and then turns off.
 - The new `reset()` method is what R calls.
- ``obstacle.py``: I added a `hit` flag so a wall that already used up your shield can't kill you on the next frame.
- ``renderer.py``: a blue circle shows around the helicopter while the shield is active. The HUD also shows distance and "SHIELD ON".

For the submission, record the "before" video from your original zip and the "after" video from this one. Show a crash, the score, a restart, and the shield taking one hit.

Attached: 08_helicopter-main-completed.zip